



Tiny Telemetry v1

Team Members:

Youssef Moustafa Elsayed	22P0047
Mostafa Fouad	22P0077
Nour Eldeen Ahmed Rehab	22P0040
Abdelrahman Mostafa	22P0053
Mohamed Yasser Khallaf	22P0245
Ibrahim Shaker Hammad	22P0056



Table of Contents

1. Introduction.....	3
1.1.Motivation.....	3
1.2.Assumptions and Constraints	3
2. Protocol Architecture	3
2.1.Entities and Roles	4
2.2.Finite State Machine (FSM)	4
3. Message Formats	5
3.1.Message Types	6
3.2.Flags Field.....	6
3.3.Payload Formats.....	6
4. Communication Procedures.....	7
4.1.Session Establishment	7
4.2.Data Transmission Modes	7
4.3.Keep Alive Mechanism.....	7
5. Reliability & Performance Features	8
5.1.Buffering and Reordering	8
5.2.Duplicate Suppression	8
5.3.Gap Detection	8
5.4.Data integrity (Checksum)	9
5.5.Selective Reliability.....	9
5.6.Batching Optimization.....	9
6. Experimental Evaluation Plan	9
6.1.Test Scenarios	9
6.2.Metrics.....	9
7. Example Use Case Walkthrough	10
8. Limitations & Future Work.....	10
8.1.Known limitations	10
8.2.Future Work.....	10
9. References	10

1. Introduction

This document specifies a lightweight IoT Telemetry Protocol (ITP) designed for sensor devices that periodically send temperature and humidity data to a central collector (server) using UDP. The goal is to provide simple, low-overhead transport for periodic sensor readings while maintaining reliability metrics such as sequence tracking and duplicate suppression.

1.1. Motivation

Existing protocols like MQTT and CoAP provide reliable messaging but add connection overhead, acknowledgments, and session management unsuitable for small embedded IoT nodes. This protocol focuses on *baseline data collection over UDP*, trading delivery guarantees for simplicity and low latency.

1.2. Assumptions and Constraints

- Communication is one-way: devices (clients) → collector (server).
- Each device has a unique Device ID assigned during startup.
- Sequence numbers increase per device and are used for detecting duplicates and missing packets.

2. Protocol Architecture

The enhanced protocol architecture adopts a stateful telemetry model overlaid on the connectionless UDP transport, where the server functions as a context-aware coordinator for multiple concurrent clients. The server enforces a logical session lifecycle, requiring an explicit handshake to establish a trusted communication channel before data processing begins. In this capacity, the server maintains distinct state vectors for each connected device, enabling it to track sequence progression, reorder out-of-sequence packets via a jitter buffer, and validate data integrity before permanent storage. Additionally, the server employs a passive liveness mechanism to govern session duration, automatically reclaiming resources from unresponsive clients while guaranteeing the delivery of critical priority events through selective application-layer acknowledgments.



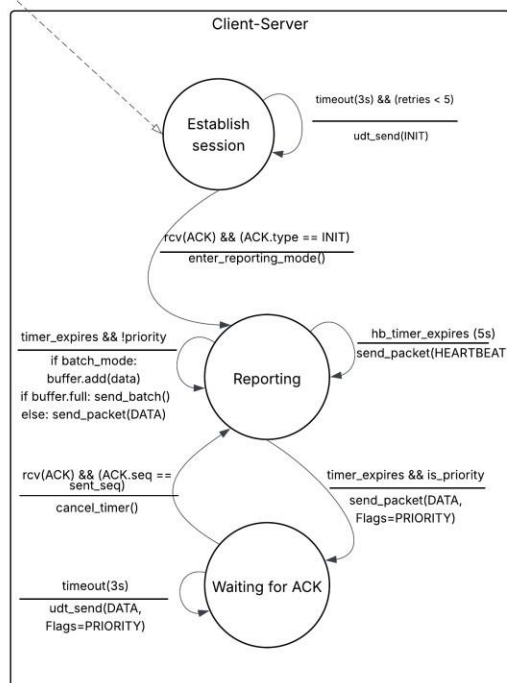
2.1. Entities and Roles

The protocol involves two main entities:

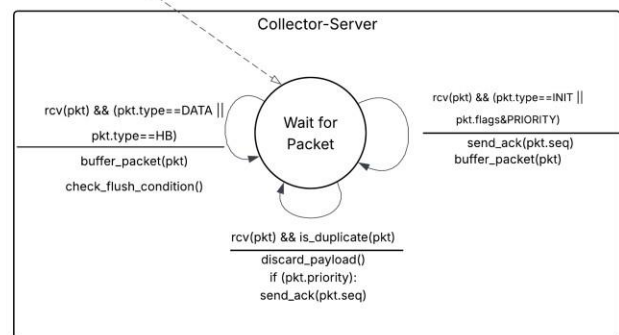
- **Client(s):** A state-machine-driven entity. It operates in two main modes:
 - Normal Mode: Sends data every 1.0s.
 - Batch Mode: Buffers N = 5 readings and flushes them as a single payload to reduce CPU interrupts.
- **Server:**
The server listens to a specific UDP port for incoming packets from all clients.

2.2. Finite State Machine (FSM)

Client 0
Client 1
Client 2
.
.
.
.
.
Client



Server (Collector)





3. Message Formats

In any communication protocol, defining a consistent and well-structured message format is essential for reliable data exchange. Proper formatting allows both the sender and receiver to correctly interpret the transmitted data, ensuring synchronization and reducing ambiguity. By separating the header (which contains metadata such as version, message type, device ID, and sequence number) from the payload (which carries the actual sensor readings like temperature and humidity), the protocol achieves modularity and extensibility. This structure also simplifies parsing, error detection, and future upgrades without breaking backward compatibility.

The previous paragraph shows that our message is divided into two main components:

- Header
- Payload

The protocol uses fixed-length header encoded in network byte order (big-endian).

Tiny Telemetry v1 Header table (total 12 bytes)

Header Field	Size (bits)	Description
Version	4	protocol version
MsgType	4	INIT or DATA OR HEARTBEAT OR ACK
DeviceID	16	Unique ID for each sensor
SeqNum	16	Order of sent packets (0 for INIT usually)
Timestamp	32	Current Unix timestamps (seconds)
Flags	8	Operational flags
Checksum	16	16-bit One's Complement Checksum (Header + Payload).



Telemetry Payload (4 bytes)

Payload Field	Size (bits)	Description
Temperature	16	Sends Accurate Temperatures to our server
Humidity	16	Sends Accurate Humidity to our server

Total Message Size: 12 (header) + 4 (payload) = 16 bytes

3.1. Message Types

The Type field (lower 4 bits of the first byte) defines the payload:

Type 0 INIT (no payload)

- Sent once when the client starts to announce its presence.

Type 1 DATA (4 Byte Payload)

- Contains the sensor readings packed as !h H (Signed Short, Unsigned Short).
- Temperature: 2 bytes (Scaled x10, e.g., 200 = 20.0°C).
- Humidity: 2 bytes (Scaled x10, e.g., 450 = 45.0%).

Type 2 HEARTBEAT (no payload)

- Sent periodically every 5s to indicate liveness when no data is changing

TYPE 3: ACK (No Payload)

- Sent by the Server to confirm receipt of INIT or Priority packets.

3.2. Flags Field

- **Bit 0:** Batch Mode If set, payload contains multiple readings.
- **Bit 1:** Priority. If set, indicates a significant data change. The Client requires an ACK for this packet.

3.3. Payload Formats

- **Normal Data:** 4 Bytes. [Temp (2B)] [Humidity (2B)] (Scaled x10).
- **Batch Data:** Variable. [Count (1B)] + N * [Temp (2B)] [Humidity (2B)].



4. Communication Procedures

This section defines the Hybrid Reliability model. The protocol mixes connectionless streaming for standard data with connection-oriented confirmed delivery for critical events.

4.1. Session Establishment

1. **INIT Request:** The Client sends a Type 0 packet. It enters a blocking wait state.
2. **Server Response:** Upon receiving a valid INIT, the Server immediately replies with a **Type 3 ACK** containing the same Sequence Number.
3. **Retries:** If the Client does not receive an ACK within the timeout, it retries up to **5 times**. If all fail, the Client terminates.

4.2. Data Transmission Modes

- **Standard Mode (Best Effort):** For small data changes, the Client sends Type 1 packets and *does not* wait for an ACK. This maximizes throughput and minimizes battery usage.
- **Priority Mode (Reliable):** If the sensor detects a large sudden change, it sets the Priority Flag (bit 2).
 - The Server must ACK this packet.
 - The Client waits for the ACK and retransmits, if necessary (Stop-and-Wait logic), ensuring critical alerts are never lost.
- **Batch Mode (High Efficiency):** To reduce CPU interruptions and header overhead, the Client can buffer readings. When the buffer fills, it flushes them in a single Type 1 packet with the Batch Flag (bit 0) set

4.3. Keep Alive Mechanism

To maintain visibility during idle periods or to verify connectivity separate from data flow:

- The Client tracks the time since the last heartbeat.
- If $\text{Time_Elapsed} \geq 5$ seconds, the Client sends a HEARTBEAT (Type 2) packet.
- This ensures the Server logs the device's presence even if data packets are not being sent.



5. Reliability & Performance Features

To mitigate the inherent unreliability of UDP while preserving the bandwidth efficiency required for IoT, the protocol employs a Hybrid Reliability Architecture. This design combines selective application-layer acknowledgments (ARQ) for critical control packets with robust server-side reconstruction logic for the standard high-volume telemetry streams.

5.1. Buffering and Reordering

- **Mechanism:** Incoming packets are held in a packet_buffer list rather than being written to disk immediately.
- **Flush Condition:** The buffer is processed ("flushed") to the CSV log when it reaches a capacity of $N=100$ packets or when a timeout event occurs.
- **Sorting:** Before logging, the buffer is sorted by Sender Timestamp. This ensures that even if packets arrive out of order (e.g., due to network jitter), they are recorded in the timeline they were created.

5.2. Duplicate Suppression

The Server maintains a seen_sequences set for each device.

- **Logic:** Before writing a packet to the CSV, the Server checks: if seq_num in seen_sequences.
- **Action:** If true, the packet is flagged as a DUPLICATE and ignored. This prevents replayed packets from corrupting the data logs.
- **Priority Exception:** If the duplicate packet is marked **Priority (Bit 2)**, the Server re-transmits the ACK before discarding the payload. This ensures that if the original ACK was lost, the Client successfully receives confirmation and ceases retransmission.

5.3. Gap Detection

The Server tracks the last_seq processed for each device.

- **Logic:** When processing a sorted batch, the Server calculates: $\text{expected} = \text{last_seq} + 1$.
- **Detection:** If $\text{current_seq} \neq \text{expected}$, the Server calculates the gap size and logs a warning: "[SERVER] sequence gap... missing X packets". This allows for passive monitoring of Packet Loss Rate (PLR).



5.4. Data integrity (Checksum)

To detect corruption over the wire (for example: bit flips), every packet includes a 16-bit Internet Checksum (One's Complement sum).

- **Implementation:** The sender calculates the checksum over the Header + Payload (setting the checksum field to 0 during calculation).
- **Verification:** The receiver recalculates the sum. If the result does not match the header field, the packet is silently discarded (CHECKSUM_MISMATCH log).

5.5. Selective Reliability

The protocol avoids the overhead of TCP by only acknowledging critical data.

- **Rationale:** Losing one standard temperature reading is acceptable. Losing the "Device Started" (INIT) signal or a "Fire Detected" (High Delta) signal is not.
- **Mechanism:** The wait_for_ack function is only invoked when msg_type == 0 (INIT) or flags & 0x02 (Priority) is true.

5.6. Batching Optimization

Batching improves Channel Utilization by reducing the ratio of Header bytes to Payload bytes.

- **Metric:** Sending 5 readings individually requires $5 * (12 \text{ Header} + 4 \text{ Payload}) = 80$ bytes.
- **Optimization:** Sending 5 readings in a batch requires $1 * 12 \text{ Header} + 1 * 21 \text{ Payload}$. This provides a significant reduction in bandwidth usage.

6. Experimental Evaluation Plan

6.1. Test Scenarios

Baseline: Verify that the Server can multiplex traffic from multiple devices.

Packet Loss: The Server detects gaps in data streams and Client successfully detects timeouts and retransmits Priority packets.

High Jitter: Force out-of-order delivery to verify that the Server's Jitter Buffer correctly sorts data by timestamp before logging.

Packet Duplication: discards payloads to protect data integrity but still sends re-ACKS for priority packets.

Batch Mode: Client buffers 5 readings into a single payload and the Server correctly unpacks them, resulting in less overhead.

6.2. Metrics

Each Test record:

- **Packets_received (successfully)**
- **Bytes_per_report**
- **Duplicate_rate**
- **Sequence_gap_count**
- **Cpu_ms_per_repor**



7. Example Use Case Walkthrough

We tested the interaction sequence involving critical events and recovery scenarios.

Scenario: A Sensor Node detects a sudden temperature spike and transmits a Priority Packet, but the initial acknowledgment is lost due to network congestion.

Walkthrough of scenario:

Event Detection: The Sensor samples a temperature of 45.0°C (previous: 41.0°C). The delta exceeds the threshold, triggering Priority Mode.

Transmission: The Sensor sets the PRIORITY flag (Bit 1), transmits Sequence #23, and starts a 3.0s retransmission timer.

Packet Loss: The Server receives Packet #23 and transmits an ACK. However, the ACK is dropped by the network.

Timeout & Retry: The Sensor's timer expires. It increments the retry counter and retransmits Sequence #23 (Payload preserved).

Duplicate handling: The Server receives the duplicate Packet #23, detects duplicate seqnum, discards payload and send an ACK.

Recovery: The Sensor receives the retransmitted ACK, cancels the timer, and resumes Standard Mode.

8. Limitations & Future Work

While ITP fulfills the core requirements for reliable telemetry, several design constraints are acknowledged for future iterations.

8.1. Known limitations

- Security: The protocol currently operates in cleartext. It lacks encryption (TLS/DTLS) and authentication, making it prone to spoofing attacks.
- Flow Control: There is no mechanism for the Server to signal the Client to slow down. A massive number of concurrent sensors could overwhelm the Server's socket buffer.

8.2. Future Work

- DTLS Integration: Implementing Datagram Transport Layer Security to provide integrity.
- Adaptive Rate Control: Modifying the Client FSM to dynamically adjust the sampling rate based on valid ACK reception rates.
- Command Downlink: allow the Server to send configuration commands to the Client via the ACK payload.

9. References

- RFC 768 – User Datagram Protocol (UDP)
- Tanenbaum & Wetherall, Computer Networks, Pearson
- Kurose & Ross, Computer Networking: A Top-Down Approach
- R. Braden, Ed., "Requirements for Internet Hosts Communication Layers,"